SEMANTIC CACHING LAYER FOR LLM APIs

Middleware That Serves Semantically Similar Requests From Cache

Project 8 of the AI Engineering Portfolio | Build Guide & Interview Companion

Author: Anas Amiar

"Users phrase the same question a hundred ways, and every phrasing is a fresh paid API call. Cache on meaning, not text — and measure the threshold tradeoff with data instead of guessing."

How to Use This Guide

This guide documents the project exactly as it was built: a short pitch, the tech stack, a phase-by-phase build order with reasoning for why each phase comes before the next, and the talking points worth memorizing before an interview. Read top to bottom to learn the project from scratch, or jump to the "Interview Talking Point" box at the end for a quick pre-call refresher.

What You're Building

A caching middleware between an application and any LLM provider. Every prompt is embedded; incoming requests semantically close to an already-answered one get the cached response in ~2ms instead of a ~1-second paid API call. Cache entries are scope-isolated (system prompt + model + temperature hashed into the key), TTLs are assigned by prompt content, and near-misses are logged for tuning.

Load test headline: **89.7% hit rate, 89.7% cost savings, P50 latency 2.6ms** on 300 requests with realistic repeat traffic. And the deeper result — the threshold tradeoff measured with ground-truth intent labels: at 0.25 similarity, 22% of cache hits served the WRONG answer; at 0.40, wrong hits drop to zero while half the traffic stays cached.

WHY THIS PROJECT LANDS INTERVIEWS

Every company running LLMs at scale has the same problem: redundant API calls burning money and adding latency. Building a semantic cache proves you think about infrastructure efficiency, not just model accuracy — and it's the kind of system whose ROI an engineering manager understands in one sentence. The wrong-hit measurement is the differentiator: most candidates would report hit rate and stop; measuring when hits become wrong answers is the production mindset.

Tech Stack

Layer	Tool / Library	Purpose
Language	Python 3.11+	Core implementation
Data validation	Pydantic v2	CacheEntry, LookupResult, CacheStats contracts
Embeddings (mock)	Bag-of-words TF + cosine	Swap embed() for a real embedding API
Store (mock)	In-memory + scope keys	Production: Redis + RedisVL or Qdrant
Cache policy	Content-based TTL tiers	Time-sensitive 1h / stable 24h / creative: never
Scope isolation	SHA-256 of (system, model, temp)	No cross-contamination between use cases
Load test	Simulated clock	Hours of traffic replayed in milliseconds
Tuner	Labeled traffic sweep	Hit rate vs. wrong-hit rate per threshold

The Core Flow

cached_completion(prompt) → embed → nearest neighbor within scope → hit (similarity ≥ threshold, TTL valid): return in ~2ms → else: call provider (~900ms, $0.002), cache under policy TTL, log near-misses.

Step-by-Step Build Guide

Six components in dependency order: the store with scope isolation first (contamination must be impossible by construction), then TTL policy, labeled traffic, the proxy, the threshold tuner, and the load test.

PHASE 1: The Cache Store with Scope Isolation *Day 1-3*

Goal: A similarity-lookup store where cross-use-case contamination is impossible.

- CacheEntry keyed by scope: SHA-256 hash of (system prompt, model, temperature).
- Lookup: embed the prompt, nearest cosine neighbor WITHIN the scope only.
- Hit if similarity ≥ threshold and TTL valid; near-miss logged if within 0.10 below the threshold.
- invalidate_scope(): system prompt changed? One call drops the whole scope.

Deliverable: SemanticCache with lookup/put/invalidate + near-miss log.

PHASE 2: The TTL Policy *Day 3-4*

Goal: Not every answer should live equally long — some shouldn't be cached at all.

- Time-sensitive prompts (today/current/latest/weather/price) → 1 hour TTL.
- Stable factual prompts → 24 hours.
- Creative prompts (write/poem/story/brainstorm) → TTL 0: never cached — replaying creative output is a bug, not a feature.

Deliverable: ttl_for_prompt() — content-based TTL tiers.

PHASE 3: Labeled Traffic + The Drop-In Proxy *Day 4-6*

Goal: Realistic test traffic with ground truth, and the call apps make instead of the LLM.

- Traffic: paraphrase groups that SHOULD share entries, plus traps — same vocabulary, different intent ('refund my subscription' vs 'cancel my subscription').
- Every query carries an intent label — the ground truth for wrong-hit measurement.
- cached_completion(): ~2ms on hits, simulated ~900ms + $0.002 on misses.
- Caller-supplied clock: TTL logic is testable, load tests run in milliseconds.

Deliverable: data/traffic.py + cache/proxy.py.

PHASE 4: The Threshold Tuner *Day 6-9*

Goal: Quantify the core tradeoff: savings vs. serving wrong answers.

- Sweep thresholds 0.25 → 0.90 over the labeled traffic.
- Per threshold: hit rate AND wrong-hit rate (hit served an entry with a different ground-truth intent).
- Result: 0.25 → 56% hits / 22% wrong; 0.40 → 50% hits / 0% wrong; 0.90 → 12% hits / 0% wrong.

Deliverable: python3 -m cache.tuner — the tradeoff table.

PHASE 5: The Load Test *Day 9-11*

Goal: The headline numbers, produced under realistic repeat traffic.

- 300 requests sampled with repetition from the traffic pool, simulated inter-arrival times.
- Report: hit rate, cost vs. no-cache baseline, latency P50/P95, near-miss count.

Deliverable: 89.7% hit rate, 89.7% savings, P50 2.6ms vs ~1s uncached.

Demo Results

Similarity threshold	Hit rate / Wrong-hit rate
0.25	56.2% hits — but 22.2% of hits served the WRONG answer
0.30	50.0% hits / 12.5% wrong
0.40 (sweet spot)	50.0% hits / 0.0% wrong
0.60	31.2% hits / 0.0% wrong
0.90	12.5% hits / 0.0% wrong — safe but barely caching
Load test @ 0.60	89.7% hit rate, 89.7% cost savings, P50 2.6ms

Architecture Decisions

Why measure wrong-hit rate instead of just hit rate?

Hit rate alone is a vanity metric — you can get 100% by setting the threshold to zero and serving garbage. The failure mode of a semantic cache is silently serving a wrong answer, which is strictly worse than a miss. Ground-truth intent labels make that failure measurable and the threshold choice data-driven.

Why scope isolation in the cache key?

The same user prompt under a different system prompt is a different request. A support bot and a marketing bot both asking 'summarize this' must not share answers. Hashing (system prompt, model, temperature) into the key makes an entire class of contamination bugs impossible by construction.

Why a simulated clock?

TTL expiry is time-based logic; testing it against the wall clock means slow, flaky tests. Passing 'now' explicitly makes TTL behavior instantly testable and lets the load test replay hours of traffic in milliseconds.

Deliberately Out of Scope for v1

- FastAPI proxy mirroring the OpenAI API contract (the cache logic is the substance).
- Real Redis/Qdrant backend — the in-memory store isolates the same interface.
- Streaming support (buffer-while-streaming on misses, cache only complete responses).
- Adaptive per-request-type thresholds learned from feedback.
- Prometheus/Grafana dashboards — stats are computed, export is plumbing.

INTERVIEW TALKING POINT

"What was the hardest design decision?"

Refusing to report hit rate without wrong-hit rate. The uncomfortable truth about semantic caching is that its failure mode is silently serving a wrong answer — strictly worse than a cache miss, and invisible unless you build the instrumentation to see it. Labeling the traffic with ground-truth intents made the failure measurable, and the threshold sweep found the operating point where savings stay high and wrong hits go to zero. Building the instrumentation to see your own failure mode is the actual engineering in this project.

What I'd Build Next

- Real embeddings + Redis/RedisVL backend for sub-millisecond lookups at scale.
- The FastAPI drop-in proxy: change your base URL, get caching with zero code changes.
- Streaming support with buffer-while-streaming on misses.

- Adaptive thresholds per request type, learned from the near-miss log and feedback.

Companion Projects in This Portfolio

Same mission as the LLM Cost Autopilot (Project 2) — cut LLM spend without hurting quality — attacked from the opposite side: the Autopilot routes requests to cheaper models; the cache eliminates repeat requests entirely. A production stack runs both: cache first, route on miss.

Repository: github.com/Anas-Amiar/Project-8-semantic-cache